پاسخ مسئله‌ی ۱.

الف

چون همهٔ هدها هایprojection یکسان دارند، تمام هدها خروجی یکسانی تولید می‌کنند:

$$W_i^Q = W^Q,\ W_i^K = W^K,\ W_i^V = W^V \quad \forall i$$
$$\implies \text{head}_i = \text{Attention}(QW^Q, KW^K, VW^V) =: H \quad \forall i$$

پس هر h هد دقیقاً همان attention را h بار محاسبه می‌کنند. خروجی لایه:

$$\begin{aligned}\text{MHA}(Q,K,V) &= \text{Concat}(H, H, \dots, H)\, W^O \\ &= \underbrace{[\,H \,|\, H \,|\, \cdots \,|\, H\,]}_{h\text{copies}}\, W^O\end{aligned}$$

با افراز سطری $W^O \in \mathbb{R}^{(h\,d_k)\times d_{\text{model}}}$ به h بلوک $W^O = \begin{bmatrix} W_{(1)}^O \\ \vdots \\ W_{(h)}^O \end{bmatrix}$، خاصیت ضرب–Concat بلوکی می‌دهد:

$$\text{MHA}(Q,K,V) = \sum_{i=1}^{h} H\, W_{(i)}^O = H\Big(\sum_{i=1}^{h} W_{(i)}^O\Big) = H\,\widetilde{W}^O$$

که در آن $\widetilde{W}^O := \sum_{i=1}^{h} W_{(i)}^O \in \mathbb{R}^{d_k \times d_{\text{model}}}$. پس خروجی همان یک attention single-head است که با یک projection مؤثر $\widetilde{W}^O$ ترکیب شده.

توجه کنید این لزوماً مضرب اسکالر H نیست؛ $\widetilde{W}^O$ یک ماتریس دلخواه است و می‌تواند ساختار غیربدیهی داشته باشد. تنها در حالت خاصِ $\widetilde{W}^O = c\,I$ به مقیاس ساده تقلیل می‌یابد.

نتیجه: قدرت بیانی Multi-Head نه از تعدد هدها، بلکه از diversity بین آن‌ها می‌آید. هدهای یکسان rank ترکیب را افزایش نمی‌دهند و کل لایه تا یک projection خطی به یک هد فرومی‌پاشد؛ پس تنوع هاsubspace شرط لازمِ بهره‌بردن از چندهد بودن است.

ب

معیار اهمیت تک‌هدی را از طریق حساسیت loss task به حذف هد تعریف می‌کنیم (به سبک Michel et al. 2019)، با gate $\xi_i \in \{۰, ۱\}$ روی هد i:

$$
\begin{aligned}
I_i &= \mathbb{E}_x \left| \mathcal{L}(x \mid \xi_i = ۱) - \mathcal{L}(x \mid \xi_i = ۰) \right| \\
&\approx \mathbb{E}_x \left| \frac{\partial \mathcal{L}}{\partial \xi_i} \right| = \mathbb{E}_x \left| H_i^{\top} \frac{\partial \mathcal{L}}{\partial H_i} \right|
\end{aligned}
$$

که از بسط مرتبهٔ اول $\mathcal{L}$ حول $\xi_i = ۱$ به دست می‌آید و در یک پاس backward برای همهٔ هدها قابل محاسبه است.

برای جنبهٔ افزونگی، شباهت زوجی هدها را با $R(i,j) = \cos(\mathrm{vec}(A_i), \mathrm{vec}(A_j))$ می‌سنجیم. هدی که هم اهمیت پایین دارد و هم با هدی دیگر به‌شدت همبسته است، نامزد حذف است. معیار ترکیبی اولویتِ حذف:

$$
\mathrm{score}(i) = \underbrace{I_i}_{\text{اهمیت}} - \lambda \underbrace{\max_{j \neq i,\, j \notin \text{pruned}} R(i,j)}_{\text{افزونگی}}
$$

هدها را بر حسب score صعودی مرتب کرده و $h - k$ هد با کمترین مقدار را حذف می‌کنیم. عبارت دوم تضمین می‌کند از هر خوشهٔ هدهای مشابه فقط یک نماینده باقی بماند (نه آنکه چند نسخهٔ مهمِ مشابه همگی نگه‌داشته شوند).

ج

متغیر دروازه‌ای $z_i \in [۰, ۱]$ را با تابع هزینه‌ای سه‌بخشی آموزش می‌دهیم:

$$
\mathcal{J} = \underbrace{\mathcal{L}_{\text{task}}(z)}_{\text{(۱) عملکرد}} + \alpha \underbrace{\sum_{i=۱}^{h} z_i}_{\text{(۲) تنکی}} + \beta \underbrace{\sum_{i<j} z_i\, z_j\, R(i,j)}_{\text{(۳) افزونگی}}
$$

نقش هر جمله:

- $\mathcal{L}_{\text{task}}(z)$: افتِ عملکرد را جریمه می‌کند $\Rightarrow$ z_i روی هدهای مفید را بالا نگه می‌دارد.
- $\alpha \sum_i z_i$ $(\ell_۱)$: فشارِ تنکی $\Rightarrow$ z_i ها را به سمت صفر می‌راند (حذف هدهای کم‌اهمیت).
- $\beta \sum_{i<j} z_i z_j R(i,j)$: جریمهٔ زوجی $\Rightarrow$ نگه‌داشتن همزمان دو هدِ مشابه را گران می‌کند.

از آنجا که هدف بردارِ تنک می‌خواهد، $\ell_۱$ مناسب‌تر از $\ell_۲$ است؛ و برای آنکه gate واقعاً به صفر/یک نزدیک شود می‌توان z_i را با Hard-Concrete پارامتری کرد تا $\sum_i z_i$ تخمینی از $\|z\|_۰$ شود.

انتخاب k پس از آموزش: هدها را بر حسب z_i نزولی مرتب می‌کنیم. مقادیر z_i معمولاً قطبیده می‌شوند (نزدیک ۰ یا ۱)؛ شکافِ طبیعی در طیف مرتب‌شدهٔ z_i مرز را مشخص می‌کند و k را برابر تعداد هدهای بالای آستانه (مثلاً $z_i > ۰/۵$) می‌گیریم — یعنی k به‌جای آنکه از پیش ثابت شود، از خودِ داده استخراج می‌گردد.

تحلیل دو هدِ کاملاً یکسان ($R(i,j) = ۱$، رفتار همسان): جملهٔ افزونگی روی این زوج برابر $\beta\, z_i z_j$ است که با نگه‌داشتن همزمانِ هر دو ($z_i =$

$z_j = ۱$) بیشینه و برابر β می‌شود. اما چون رفتارشان یکسان است، خروجی $z_i H_i + z_j H_j = (z_i + z_j)H$ تنها به مجموع $z_i + z_j$ وابسته است؛ پس همان عملکرد با $(z_i, z_j) = (۱, ۰)$ نیز حاصل می‌شود بی‌آنکه $\mathcal{L}_{\text{task}}$ تغییر کند، حال‌آنکه هم جریمهٔ تنکی ($-\alpha$) و هم جریمهٔ افزونگی ($-\beta$) کاهش می‌یابند:

$$\mathcal{J}(۱, ۱) - \mathcal{J}(۱, ۰) = \alpha + \beta > ۰$$

بنابراین جریمهٔ افزونگی $\mathcal{J}$ را در حالتِ «نگه‌داشتن هر دو» اکیداً بزرگ‌تر می‌کند بی‌آنکه نفعی در task بدهد؛ پس راه‌حل بهینه یکی از این دو هدِ تکراری را خاموش می‌کند و نگه‌داشتن همزمان هر دو نامطلوب است.

پاسخ مسئله‌ی ۲.

الف

نمادگذاری: $\mathcal{S} = \text{Sublayer}$، $\Gamma = \text{diag}(\gamma)$، $\hat{x} = \frac{y - \mu \mathbf{1}}{\sigma}$ و d بُعد ویژگی.

برای Post-LN، با $y_l = x_l + \mathcal{S}(x_l)$ و $x_{l+1} = \text{LN}(y_l)$:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_{l+1}} J_{\text{LN}}(y_l) \left(I + \frac{\partial \mathcal{S}}{\partial x_l} \right)$$

ژاکوبین LayerNorm:

$$J_{\text{LN}} = \frac{\partial \text{LN}(y)}{\partial y} = \frac{\Gamma}{\sigma} \Big(I - \tfrac{1}{d} \mathbf{1}\mathbf{1}^\top - \tfrac{1}{d} \hat{x}\hat{x}^\top \Big)$$

اثر اندازه: ضریب γ/σ نُرم گرادیان را تغییر می‌دهد (با بزرگ‌شدن σ، گرادیان کوچک می‌شود). اثر جهت: عملگر تصویرِ $P = I - \frac{1}{d}\mathbf{1}\mathbf{1}^\top - \frac{1}{d}\hat{x}\hat{x}^\top$ مؤلفه‌های در راستای $\mathbf{1}$ (میانگین) و $\hat{x}$ (خودِ ورودیِ نرمال‌شده) را حذف می‌کند؛ پس $J_{\text{LN}} \neq cI$، رتبه را کاهش و جهت گرادیان را تغییر می‌دهد.

در عمق L، گرادیانِ مسیر میان‌بر نیز در هر لایه در J_{LN} ضرب می‌شود:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=0}^{L-1} J_{\text{LN}}^{(l)} \Big(I + \tfrac{\partial \mathcal{S}^{(l)}}{\partial x_l} \Big)$$

حاصل‌ضربِ L ژاکوبین با ضریب γ/σ رشد یا افت هندسی (انفجار/محوشدگیِ گرادیان) ایجاد می‌کند.

برای Pre-LN، با $x_{l+1} = x_l + \mathcal{S}(\text{LN}(x_l))$:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_{l+1}} \Big(I + J_{\mathcal{S}}^{(l)} J_{\text{LN}}^{(l)} \Big)$$
$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=0}^{L-1} \Big(I + J_{\mathcal{S}}^{(l)} J_{\text{LN}}^{(l)} \Big) = \frac{\partial \mathcal{L}}{\partial x_L} \Big(I + \textstyle\sum_l J_{\mathcal{S}}^{(l)} J_{\text{LN}}^{(l)} + \cdots \Big)$$

جملهٔ پیشروِ I یک مسیرِ همانیِ بدونِ نرمال‌سازی است؛ گرادیان بدون ضرب‌شدنِ مکرر در J_{LN} به همهٔ لایه‌ها می‌رسد، پس مقیاسِ گرادیان مستقل از عمق و کران‌دار می‌ماند.

نتیجه: نبودِ حاصل‌ضربِ هندسیِ ژاکوبین‌ها در Pre-LN، گرادیان را در هر عمق پایدار نگه می‌دارد و آموزشِ مدل‌های عمیق پایدارتر می‌شود. در Post-LN گرادیانِ لایه‌های نزدیک خروجی در آغاز بسیار بزرگ است و نرخ یادگیریِ زیاد، تخمین‌های وفقیِ Adam را بی‌ثبات می‌کند؛ warmup برای مهارِ همین گرادیانِ بزرگِ اولیه لازم است. چون Pre-LN مقیاسِ گرادیان را از ابتدا کران‌دار و مستقل از عمق می‌سازد، نیاز به warmup کاهش می‌یابد یا حذف می‌شود.

ب

(۱) عمقِ مؤثر: در Pre-LN به‌دلیلِ مسیرِ همانیِ غالب، $x_L = x_0 + \sum_l \mathcal{S}_l(\text{LN}(x_l))$، واریانسِ جریانِ باقی‌مانده با عمق انباشته می‌شود ($\text{Var}(x_l) \sim l$). پس سهمِ نسبیِ هر زیرلایه $\frac{\|\mathcal{S}_l\|}{\|x_l\|} \sim \frac{1}{\sqrt{l}}$ کوچک می‌شود و لایه‌های عمیق تقریباً همانی عمل می‌کنند؛ یعنی عمقِ مؤثرِ تبدیلِ

ویژگی کم می‌شود و ظرفیتِ مدل کامل به‌کار نمی‌رود. در Post-LN نرمال‌سازیِ پس از هر جمع، سهمِ همهٔ لایه‌ها را هم‌مقیاس نگه می‌دارد و تبدیلِ ویژگیِ غنی‌تری می‌دهد.

(۲) توزیعِ activation ورودی به لایهٔ نهایی: آخرین عملیاتِ Post-LN یک LayerNorm است، پس ورودیِ سرِ خروجی نرمال (میانگین صفر، مقیاسِ کنترل‌شده) و خوش‌وضع است. در Pre-LN آخرین عملیات یک جمع است و activationهای نهایی نرمال‌نشده و با واریانسِ بزرگ وارد سر می‌شوند؛ این شرطِ بدتر، بهینه‌سازیِ سر و برازشِ نهایی را اندکی ضعیف‌تر می‌کند.

(۳) نقشِ نرمال‌سازی به‌عنوانِ regularization: در Post-LN نرمال‌سازی روی مسیرِ اصلیِ سیگنال در هر لایه اعمال می‌شود و با محدودکردنِ مقیاس و توزیعِ بازنمایی‌های میانی یک regularization ضمنی ایجاد می‌کند که در مقیاسِ کوچک تعمیم را بهبود می‌دهد. Pre-LN فقط ورودیِ شاخه را نرمال می‌کند و جریانِ باقی‌مانده آزاد می‌ماند؛ کمبودِ این اثرِ تنظیمی می‌تواند به کمینهٔ کمی بدتر یا بیش‌برازش بینجامد.

جمع‌بندی: Pre-LN پایداری را با هزینهٔ کاهشِ عمقِ مؤثر، بدترشدنِ وضعِ ورودیِ لایهٔ نهایی و کم‌شدنِ regularization ضمنی به‌دست می‌آورد؛ در مدل‌های کوچک که پایداری گلوگاه نیست، این هزینه‌ها غالب می‌شوند و عملکردِ نهایی اندکی پایین‌تر می‌آید.

ج

نمونه: Fixup initialization و نسخهٔ ترنسفورمریِ آن T-Fixup؛ روشِ جایگزینِ دیگر DeepNorm است.

ایدهٔ T-Fixup/Fixup: حذفِ کاملِ LayerNorm و کنترلِ مقیاس فقط از راهِ مقداردهیِ اولیه:

- آخرین لایهٔ وزنیِ هر شاخهٔ باقی‌مانده با صفر مقداردهی می‌شود تا در آغاز هر بلوک دقیقاً همانی باشد؛ پس واریانسِ سیگنالِ پیش‌رو در عمق حفظ می‌شود.
- سایر وزن‌ها با ضریبِ $L^{-۱/(۲m-۲)}$ (m تعدادِ لایه‌های شاخه) کوچک می‌شوند تا تغییرِ خروجی پس از یک گامِ بهینه‌سازی به‌ازای هر بلوک $\Theta(۱/L)$ و در کل $\Theta(۱)$ بماند.

DeepNorm: مسیرِ باقی‌مانده را بزرگ‌نمایی می‌کند، $x_{l+۱} = \mathrm{LN}(\alpha x_l + \mathcal{S}(x_l))$ با $\alpha > ۱$، و برخی وزن‌ها را با $\beta < ۱$ کوچک می‌کند تا اندازهٔ به‌روزرسانیِ مدل مستقل از عمق کران‌دار شود.

توضیحِ نظری: کاری که نرمال‌سازی انجام می‌دهد، تأمینِ سه نامتغیّرِ مقیاسی در سراسرِ عمق است:

$$(۱)\mathrm{Var}(x_l) = \Theta(۱), \qquad (۲)\left\|\tfrac{\partial \mathcal{L}}{\partial x_l}\right\| = \Theta(۱), \qquad (۳)\|\Delta x_L\|_{\text{step per}} = \Theta(۱)$$

نرمال‌سازی این‌ها را به‌صورتِ پویا و در زمانِ اجرا برقرار می‌کند، اما این‌ها صرفاً شروطِ مقیاس‌اند. اگر همین شروط به‌صورتِ ایستا تضمین شوند—صفرکردنِ شاخه‌ها واریانسِ پیش‌رو را حفظ می‌کند (۱)، و کوچک‌سازیِ وابسته به عمقِ وزن‌ها نُرمِ گرادیان و اندازهٔ به‌روزرسانی را کران‌دار می‌کند (۲) و (۳)—آنگاه چشم‌اندازِ بهینه‌سازی به همان اندازه خوش‌وضع است و آموزش بدونِ هیچ نرمال‌سازیِ زمان‌اجرا پایدار می‌ماند. پس نرمال‌سازی شرطِ کافی برای آموزش‌پذیری است، نه لازم؛ آنچه لازم است حفظِ همین نامتغیّرهای مقیاس و خوش‌وضعی است که از راهِ مقداردهیِ اولیه و مقیاس‌بندیِ مناسبِ شاخهٔ باقی‌مانده نیز به‌دست می‌آید. تطبیقِ تجربیِ DeepNorm/T-Fixup/Fixup با خطوطِ پایهٔ نرمال‌شده همین را تأیید می‌کند.

پاسخ مسئله‌ی ۳.

الف

ابتدا ماتریس $A_i(k)$ را می‌یابیم. می‌دانیم که:

$$p_i(t+k) = \begin{bmatrix} \sin(\omega_i(t+k)) \\ \cos(\omega_i(t+k)) \end{bmatrix}$$

با بسط توابع مثلثاتی جمع زوایا:

$$\sin(\omega_i t + \omega_i k) = \sin(\omega_i t)\cos(\omega_i k) + \cos(\omega_i t)\sin(\omega_i k)$$
$$\cos(\omega_i t + \omega_i k) = \cos(\omega_i t)\cos(\omega_i k) - \sin(\omega_i t)\sin(\omega_i k)$$

این دو را به‌صورت ماتریسی بازنویسی می‌کنیم:

$$p_i(t+k) = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{bmatrix} \sin(\omega_i t) \\ \cos(\omega_i t) \end{bmatrix}$$

از این رو ماتریس خطی مستقل از t به‌دست می‌آید:

$$A_i(k) = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix}$$

این یک ماتریس دوران به اندازه زاویه $-\omega_i k$ است. حال خاصیت جمع‌پذیری را نشان می‌دهیم. با ضرب دو ماتریس دوران:

$$A_i(k_1)A_i(k_2) = \begin{bmatrix} \cos(\omega_i k_1) & \sin(\omega_i k_1) \\ -\sin(\omega_i k_1) & \cos(\omega_i k_1) \end{bmatrix} \begin{bmatrix} \cos(\omega_i k_2) & \sin(\omega_i k_2) \\ -\sin(\omega_i k_2) & \cos(\omega_i k_2) \end{bmatrix}$$

درایه بالا_چپ حاصل‌ضرب برابر است با:

$$\cos(\omega_i k_1)\cos(\omega_i k_2) - \sin(\omega_i k_1)\sin(\omega_i k_2) = \cos(\omega_i(k_1+k_2))$$

و درایه بالا_راست:

$$\cos(\omega_i k_1)\sin(\omega_i k_2) + \sin(\omega_i k_1)\cos(\omega_i k_2) = \sin(\omega_i(k_1+k_2))$$

با محاسبه مشابه دو درایه دیگر، نتیجه می‌شود:

$$A_i(k_1)A_i(k_2) = \begin{bmatrix} \cos(\omega_i(k_1+k_2)) & \sin(\omega_i(k_1+k_2)) \\ -\sin(\omega_i(k_1+k_2)) & \cos(\omega_i(k_1+k_2)) \end{bmatrix} = A_i(k_1+k_2)$$

این رابطه نشان می‌دهد که جابه‌جایی‌ها در sinuosoidal positional encoding ساختار یک گروه را دارند: اثر یک جابه‌جایی به اندازه k صرفاً یک دوران در صفحه دوبعدی p_i است، و ترکیب دو جابه‌جایی متوالی معادل دوران به اندازه مجموع آن‌هاست. به بیان دیگر، نگاشت $k \mapsto A_i(k)$ یک همریختی از گروه جمعی $(\mathbb{R}, +)$ به گروه دوران‌هاست؛ همین خاصیت است که اطلاعات نسبی موقعیت را به‌صورت خطی قابل بازیابی می‌کند.

ب

برای دو موقعیت m و n، ضرب داخلی بردارهای دوبعدی را محاسبه می‌کنیم:

$$p_i(m)^\top p_i(n) = \sin(\omega_i m)\sin(\omega_i n) + \cos(\omega_i m)\cos(\omega_i n)$$

این دقیقاً بسط cos تفاضل زوایاست، پس:

$$p_i(m)^\top p_i(n) = \cos(\omega_i(m-n))$$

حال بردار کامل $PE(t)$ از الحاق همه جفت‌های دوبعدی $p_i(t)$ ساخته شده است. چون ضرب داخلی دو بردار الحاق‌شده برابر مجموع ضرب‌های داخلی بلوک‌های متناظر است:

$$PE(m)^\top PE(n) = \sum_i p_i(m)^\top p_i(n) = \sum_i \cos(\omega_i(m-n))$$

نتیجه می‌گیریم که شباهت (ضرب داخلی) دو بردار مکانی تنها تابعی از فاصله نسبی $m-n$ است و به مقادیر مطلق m و n به‌صورت جداگانه بستگی ندارد. بنابراین گرچه sinusoidal positional encoding از موقعیت‌های مطلق ساخته می‌شود، ساختار درونی آن به‌گونه‌ای است که در محاسبه شباهت، اطلاعات نسبی موقعیت به‌طور طبیعی ظاهر می‌شود.

پاسخ مسئله‌ی ۴.

پاسخ مسئله‌ی ۵.

مشخصات مدل: $L = ۲۴$ لایه، $d = ۱۰۲۴$، تعداد سرها ۱۶ =، و در مجموع ۳۵۰×۱۰^۶ پارامتر. تعداد وظایف $T = ۲۰$ است.

الف) روش full fine-tuning

در این روش برای هر وظیفه یک نسخه‌ی کامل از مدل ذخیره می‌شود، پس پارامترهای اختصاصی هر وظیفه برابر کل پارامترهای مدل است.

$$\text{کل پارامترهای ذخیره‌شده} = T \times ۳۵۰\text{M} = ۲۰ \times ۳۵۰ \times ۱۰^۶$$
$$= ۷۰۰۰ \times ۱۰^۶ = ۷ \times ۱۰^۹ = ۷\text{B}$$

یعنی در مجموع باید ۷ میلیارد پارامتر ذخیره شود.

ب) روش Prompt Tuning

در این روش مدل پایه ثابت می‌ماند و تنها یک soft prompt با طول $m = ۵۰$ که هر بردار آن بعد $d = ۱۰۲۴$ دارد، برای هر وظیفه آموزش می‌بیند.

(۱) تعداد پارامترهای آموزش‌پذیر برای هر وظیفه:

$$P_{\text{task}} = m \times d = ۵۰ \times ۱۰۲۴ = ۵۱۲۰۰ \approx ۵۱/۲\text{K}$$

(۲) کل پارامترهای اختصاصی برای ۲۰ وظیفه:

$$P_{\text{total}} = T \times m \times d = ۲۰ \times ۵۱۲۰۰ = ۱۰۲۴۰۰۰ \approx ۱/۰۲\text{M}$$

ج) روش Prefix Tuning

در این روش مدل ثابت می‌ماند و در هر لایه‌ی Transformer برای هر وظیفه، یک prefix برای بخش Key با طول $p = ۱۰$ و یک prefix برای بخش Value با طول $p = ۱۰$ اضافه می‌شود؛ هر بردار بعد $d = ۱۰۲۴$ دارد.

(۱) تعداد پارامترهای آموزش‌پذیر برای هر وظیفه: در هر لایه دو prefix (یکی Key و یکی Value) داریم، پس:

$$P_{\text{task}} = L \times (۲ \times p \times d) = ۲۴ \times (۲ \times ۱۰ \times ۱۰۲۴)$$
$$= ۲۴ \times ۲۰۴۸۰ = ۴۹۱۵۲۰ \approx ۴۹۱/۵\text{K}$$

(۲) کل پارامترهای اختصاصی برای ۲۰ وظیفه:

$$P_{\text{total}} = T \times L \times ۲ \times p \times d = ۲۰ \times ۴۹۱۵۲۰$$
$$= ۹۸۳۰۴۰۰ \approx ۹/۸۳\text{M}$$

د) نسبت کاهش ذخیره‌سازی نسبت به full fine-tuning

مبنای مقایسه، پارامترهای اختصاصی هر وظیفه است؛ در full fine-tuning این مقدار ۳۵۰M به ازای هر وظیفه است.

(۱) Prompt Tuning:

$$\text{نسبت} = \frac{۳۵۰ \times ۱۰^۶}{۵۱۲۰۰} \approx ۶۸۳۶$$

یعنی حدود ۶۸۳۶ برابر کمتر فضا (کاهش تقریباً ٪۰/۰۱۴۶ از حالت کامل).

(۲) Prefix Tuning:

$$\text{نسبت} = \frac{۳۵۰ \times ۱۰^۶}{۴۹۱۵۲۰} \approx ۷۱۲$$

یعنی حدود ۷۱۲ برابر کمتر فضا (کاهش تقریباً ٪۰/۱۴ از حالت کامل).

ه) تفاوت اصلی Prompt Tuning و Prefix Tuning

تفاوت اصلی در محل تزریق پارامترهای آموزش‌پذیر است. در Prompt Tuning تنها چند بردار به ورودی (لایه‌ی embedding) افزوده می‌شود و مدل از طریق همان لایه‌ی اول این بردارها را در سراسر شبکه منتشر می‌کند. اما در Prefix Tuning در هر لایه‌ی Transformer و به‌طور مستقیم در بخش‌های Key و Value مکانیزم attention، بردارهای پیشوند اضافه می‌شود.

Prompt Tuning:

- مزیت: تعداد پارامتر بسیار کم و ذخیره‌سازی فوق‌العاده سبک.
- محدودیت: ظرفیت بیان کمتر و معمولاً عملکرد ضعیف‌تر روی مدل‌های کوچک یا وظایف پیچیده.

Prefix Tuning:

- مزیت: نفوذ مستقیم در همه‌ی لایه‌ها، ظرفیت بیان بالاتر و عملکرد بهتر.
- محدودیت: پارامترهای بیشتر (به نسبت layerها) و پیچیدگی پیاده‌سازی بالاتر.

پاسخ مسئله‌ی ۶.

پاسخ مسئله‌ی ۷.